

Computational Linguistics 1 (2026)

Assignment 2: Rule-Based Morphology

(Item-and-Process Model)

Deadline

February 17, 2026 (Tuesday), 11:59 PM

Objective

This assignment focuses on **rule-based morphological analysis and generation** using the **Item-and-Process (IP)** model. You will design linguistically motivated rules using **regular expressions and Unicode-based processing**.

The emphasis is on:

- grouping word forms using linguistic generalizations,
- explicit surface-to-structure mapping,
- understanding overgeneration and ambiguity.

Machine learning approaches are strictly not allowed.

Important Notes

- This assignment follows a **scaffolded programming approach**.
- Around **70–80% of the code is provided**.
- This is a **rule-heavy assignment**.
- For **every rule you write**, you must clearly explain:
 - why those words are grouped together,
 - what linguistic generalization the regex or Unicode mapping captures.
- Marks are awarded for **linguistic reasoning**, not rule count.

Tasks Overview

You must complete **all five tasks**:

1. Morphological Analyzer for English
2. Morphological Generator for English

3. Unicode-Based Script Converter for an Indian Language
4. Morphological Analyzer for an Indian Language
5. Morphological Generator for an Indian Language

Important:

- **Task 3 output must be used as input for Task 4 and Task 5.**
- Do **not** write morphological rules directly on native Unicode script.

Task 1: Morphological Analyzer for English (20 Marks)

Build a rule-based morphological analyzer using Item-and-Process rules.

Sample Input / Output

```
playing → play + V + PROG
dogs → dog + N + PL
```

Provided Code Skeleton

```
import re

def analyze_english(word):
    """
    Rule-based morphological analyzer for English
    """

    if re.search(r"ing$", word):
        root = re.sub(r"ing$", "", word)
        return root, ["V", "PROG"]

    elif re.search(r"ed$", word):
        root = re.sub(r"ed$", "", word)
        return root, ["V", "PAST"]

    elif re.search(r"s$", word):
        root = re.sub(r"s$", "", word)
        return root, ["N", "PL"]

    # TODO: add more rules (e.g., -ies, -en, irregulars)

    else:
        return word, ["UNK"]
```

Requirement

- Extend the given code by adding a comprehensive set of rule-based morphological transformations that cover regular forms, orthographic variations, and common irregular cases in English.
- Explain why each regex groups those words

Task 2: Morphological Generator for English (20 Marks)

Generate surface word forms from root and feature set.

Provided Code Skeleton

```
def generate_english(root, features):  
    """  
    Rule-based morphological generator for English  
    """  
  
    if "V" in features and "PROG" in features:  
        return root + "ing"  
  
    elif "V" in features and "PAST" in features:  
        return root + "ed"  
  
    elif "N" in features and "PL" in features:  
        return root + "s"  
  
    # TODO: add more generation rules  
  
    else:  
        return root
```

Requirement

Explain how generation rules correspond to analysis rules.

Task 3: Unicode-Based Script Converter (20 Marks)

Build a Unicode-based converter that maps native Indian language script into a **standardized computational representation**.

Choose ONE Representation

1. ISO Transliteration (e.g., ISO 15919)
2. IPA (International Phonetic Alphabet)
3. WX Notation

Clearly specify your chosen standard in the README.

Example

పస్తాకాలు → pustakAlu (WX)

Provided Code Skeleton

```
def unicode_converter(text):  
    """  
    Converts native script into ISO / IPA / WX representation  
    """
```

```

output = ""

for ch in text:
    code = ord(ch)

    # Example: Telugu 'ప' -> 'p' (WX)
    if code == 0x0C2A:
        output += "p"

    # Example: Telugu vowel sign 'ు' -> 'u'
    elif code == 0x0C41:
        output += "u"

    # TODO: add more Unicode mappings

    else:
        output += ch

return output

```

Mandatory Explanation

For every Unicode mapping, explain:

- the phoneme/grapheme represented,
- why it maps to that symbol,
- why this conversion helps morphology.

Task 4: Morphological Analyzer for an Indian Language (20 Marks)

Build a morphological analyzer using **only the output of Task 3**.

Example

Input to Analyzer: pustakAlu

Output: pustaka + N + PL

Provided Code Skeleton

```

import re

def analyze_indian(word):
    """
    Morphological analyzer using converted representation
    """

    if re.search(r"Alu$", word):
        root = re.sub(r"Alu$", "", word)
        return root, ["N", "PL"]

    # TODO: add tense, case, agreement rules

```

```
else:
    return word, ["UNK"]
```

Requirement

- Extend the given code by adding a comprehensive set of rule-based morphological transformations that cover regular forms, orthographic variations, and common irregular cases
- Explain why each regex grouping is linguistically valid

Task 5: Morphological Generator for an Indian Language (20 Marks)

Generate word forms using the same representation as Task 3.

Provided Code Skeleton

```
def generate_indian(root, features):
    """
    Morphological generator for an Indian language
    """

    if "N" in features and "PL" in features:
        return root + "Alu"

    # TODO: add tense, case, agreement generation rules

    else:
        return root
```

Note

Optional: You may convert generated output back to native script, but this is **not required**.

Testing and Error Analysis

- Test each module on at least **15 words**
- Identify:
 - correct analyses,
 - over-generation cases,
 - rule conflicts or ambiguity.

Submission Instructions

Submit a single ZIP file named:

<rollnumber>_assignment2.zip

The ZIP file must contain:

- english_analyzer.py

- `english_generator.py`
- `unicode_converter.py`
- `indian_analyzer.py`
- `indian_generator.py`
- `README.pdf` / `README.md`

Evaluation Scheme

Component	Marks
English Analyzer	20
English Generator	20
Unicode Converter	20
Indian Analyzer	20
Indian Generator	20
Total	100

Academic Integrity

This assignment must be entirely your own work. Rule justification and linguistic explanation are mandatory.

Suggested Reading

- Spencer, A. (1991). *Morphological Theory: An Introduction to Word Structure in Generative Grammar*. Oxford: Blackwell Publishers.
- Beesley, K. R., & Karttunen, L. (2003). *Finite State Morphology*. Stanford, CA: CSLI Publications.
- The Unicode Consortium. (2023). *The Unicode Standard, Version 15.0 — Code Charts for Indian Scripts*. Mountain View, CA: Unicode Consortium. Available at: <https://www.unicode.org/charts/>